



**POLYTECHNIQUE
MONTRÉAL**

UNIVERSITÉ
D'INGÉNIERIE

Département de génie informatique et génie logiciel

Cours INF1900:
Projet initial de système embarqué

Travaux pratiques 7 et 8

Production de librairie statique et stratégie de débogage

Par l'équipe

No 5456

Noms:

Anis, Ranya

Ezzarouali, Mohamed-Adam

Julien, Noé

Zidi, Rym

Date:

28 octobre 2025

Partie 1 : Description de la librairie

Nous avons structuré nos classes de la manière détaillée dans le rapport afin de simplifier la réutilisation du code développé lors des TP 1, 2, 4, 5 et 6. La librairie permet de regrouper les méthodes récurrentes fréquemment utilisées, sans avoir à les redéfinir à chaque nouveau projet. Elle centralise ainsi l'ensemble des fonctions, attributs, modes de fonctionnement (CTC, PWM) et paramètres de registres au sein d'un module commun. Les classes et leurs méthodes sont désormais généralisées, offrant une utilisation plus flexible, modulaire et polyvalente, notamment pour les projets futurs tels que le projet final. Chaque classe de la librairie possède une fonction spécifique liée à un aspect précis du robot (communication, minuteries, motricité etc.). Cela assure une organisation claire du code. De plus, certaines classes interagissent entre elles. Par exemple, une classe peut utiliser les services d'une autre (comme `usart` qui est réutilisé dans `debug`).

Cette séparation permet de clarifier le code, de distinguer la logique de l'implémentation de la définition des interfaces et de favoriser la réutilisation des modules.

La séparation des fichiers est organisée comme suit :

- Fichier d'en-tête (.hpp) :
Ce fichier contient la déclaration des classes, de leurs attributs et méthodes publiques ou privées. De plus, il contient des constantes, énumérations et les fonctions nécessaires. Il définit l'interface du module, c'est-à-dire ce qui est accessible aux autres parties du programme.
- Fichier source (.cpp) :
Ce fichier contient l'implémentation des fonctions et méthodes déclarées dans le fichier d'en-tête. On y décrit le comportement concret des classes et des fonctions. C'est dans ces fichiers que se trouve les instructions spécifiques à chaque module.

1) Classe del

La classe `del` permet de gérer les DELs du robot. Elle regroupe dans une même classe la logique d'allumage des DELs (rouge, vert, éteint et ambre). Cela permet d'éviter de répéter la configuration des broches et les écritures sur les ports dans chaque programme.

La fonction `allumerAmbre()` est bloquante, car elle utilise `_delay_ms()` pour alterner rapidement entre les couleurs rouge et verte.

2) Classe interruptions

La classe `interruption` permet de configurer et d'utiliser les interruptions externes `INT0` et `INT1`, ainsi qu'une minuterie basée sur le `Timer1`. Elle regroupe dans une même classe la configuration des registres et la gestion des différents modes de déclenchement (`LowLevel`, `AnyEdge`, `FallingEdge`, `RisingEdge`).

3) Classe timer

La classe timer fournit une interface unifiée pour configurer et utiliser les minuteries matérielles du microcontrôleur ATmega324PA. Elle permet de contrôler le temps, la fréquence et le rapport cyclique (PWM) des signaux, ainsi que de générer des interruptions périodiques pour le contrôle du robot (clignotement de DEL, moteurs, mesures, etc.). Elle contient principalement les énumérations Type, Mode et Prescaler. Elle n'a pas de constructeur spécifique, car aucune initialisation n'est requise lors de la création de l'objet. Elle agit comme un regroupement logique des fonctions permettant de configurer les minuteries matérielles et de générer des signaux précis.

4) Classe usart

La classe usart (Universal Synchronous/Asynchronous Receiver Transmitter) permet d'assurer la communication série (RS-232) entre le microcontrôleur ATmega324PA du robot et un ordinateur. Elle encapsule les fonctions de configuration et de transmission UART dans une classe orientée objet (Usart). Le constructeur Usart() configure le port série à 2400 bauds, 8 bit de données cela permet au robot de communiquer avec un terminal PC via RS-232.

5) Classe memoire24CXXX

Cette classe permet de lire et d'écrire dans une mémoire EEPROM externe I²C. Elle simplifie l'accès à la mémoire en prenant en charge la gestion des registres et des cycles de communication. La communication repose sur le protocole TWI (Two-Wire Interface), qui assure la synchronisation et le transfert des données. Ces fonctions gèrent les registres TWBR, TWCR, TWDR et TWSR, ainsi que les cycles de transmission requis pour communiquer avec la mémoire.

Fonctions critiques :

- `init()` : initialise l'interface I²C sur SDA (PC1) et SCL (PC0), configure la vitesse avec TWBR et TWSR, puis appelle `choisir_banc(0)`. Appelée automatiquement par le constructeur.
- `lecture()` : lit un ou plusieurs octets depuis l'EEPROM. Fonction bloquante utilisant TWCR, TWDR et TWSR pour la gestion du transfert.
- `ecriture()` : écrit un ou plusieurs octets dans l'EEPROM. Bloquante également, car elle attend la fin du cycle d'écriture et utilise les mêmes registres de contrôle.

6) Classe can

La classe can effectue la conversion analogique-numérique sur le microcontrôleur ATmega324PA. Elle lit les signaux analogiques provenant de capteurs (luminosité, température, distance, etc.) et les convertit en valeurs numériques sur 10 bits, exploitables pour le traitement et la prise de décision. Les fichiers associés définissent la classe, la configuration et l'utilisation du convertisseur analogique-numérique (ADC) à travers les registres ADMUX et ADCSRA.

Fonctions critiques :

- Constructeur CAN() : configure la référence de tension, l'ajustement des données et le facteur de division de l'horloge via ADMUX et ADCSRA.
- Destructeur ~CAN() : désactive le convertisseur pour réduire la consommation d'énergie.
- lecture(uint8_t pos) : lance une conversion sur le canal analogique spécifié, attend la fin du processus et retourne une valeur sur 16 bits, dont les 10 bits de poids faibles contiennent la mesure.

7) Classe pwm

La classe pwm contrôle un signal de modulation de largeur d'impulsion (*Pulse Width Modulation*), utilisé pour régler la vitesse des moteurs ou l'intensité d'une DEL. Elle repose sur la configuration du Timer2 en mode Phase Correct PWM 8 bits, avec les broches PD6 et PD7 définies en sortie. Le fichier d'en-tête contient la définition de la classe et de ses méthodes publiques, tandis que le fichier source configure la minuterie et implémente les fonctions associées.

Fonctions critiques :

- Constructeur PWM() : configure les registres TCCR2A et TCCR2B, active les sorties OC2A et OC2B, et initialise le mode *Phase Correct PWM*. Fonction non bloquante mais essentielle pour activer le module.
- ajustementPwm(uint8_t ratioA, uint8_t ratioB) : modifie les registres OCR2A et OCR2B pour ajuster le rapport cyclique du signal. Fonction non bloquante, agissant en temps réel sur les sorties matérielles.
- eteindre() : met les valeurs OCR2A et OCR2B à zéro afin de désactiver le PWM.

8) Classe robotMotricite

Cette classe gère la motricité du robot en contrôlant indépendamment chaque roue. Elle centralise la gestion de la vitesse et de la direction, facilitant la réutilisation et la clarté du

code. Elle utilise la classe PWM pour le contrôle des vitesses et définit les broches de direction selon les paramètres fournis au constructeur.

Fonctions critiques :

- Avancer / Reculer / Tourner : fonctions non bloquantes et rapides, basées sur `setDirection()` et `PWM::ajustementPwm()`. Pour tourner, la direction d'une roue est inversée par rapport à l'autre, ce qui fait pivoter le robot sur lui-même.
- `setDirection(Direction gauche, Direction droite)` : modifie les valeurs en sortie selon la direction voulue.
- `arreter()` : coupe le signal PWM à l'aide de `PWM::eteindre()`.

Constructeur `RobotMotricite()` : configure les ports de direction et initialise l'objet PWM pour permettre tous les types de déplacement.

9) Classe debug

La librairie debug facilite le diagnostic et la vérification du comportement du robot. Elle fournit des outils simples pour afficher des messages de débogage, surveiller des variables, valider le fonctionnement des modules (USART, capteurs, moteurs, etc.) et identifier rapidement les erreurs logicielles ou matérielles. Elle regroupe les fonctions nécessaires à l'envoi de messages de débogage, à l'affichage de valeurs et au contrôle visuel de l'état du système, notamment par la communication série. Le constructeur `Debug()` initialise les ressources nécessaires au débogage, telles que l'initialisation du port série ou la configuration des broches utilisées pour l'affichage d'état. La classe debug peut ensuite être utilisée tout au long du développement pour tracer l'exécution du programme, confirmer la réception de données, ou vérifier le comportement des différentes librairies intégrées au projet.

Partie 2 : Décrire les modifications apportées au Makefile de départ

Modifications du Makefile de la librairie:

La première modification concerne la gestion automatique des fichiers sources. Alors que le Makefile original spécifiait manuellement un fichier source unique, notre version utilise la fonction `$(wildcard *.cpp)` pour inclure automatiquement tous les fichiers C++ présents dans le répertoire. Cette approche dynamique élimine la nécessité de modifier manuellement le Makefile lors de l'ajout de nouveaux modules à la librairie, réduisant ainsi les risques d'erreurs et facilitant la maintenance du projet.

De plus, nous avons ajouté des chemins d'inclusion. L'ajout de `LIBS= -L ../lib -l:librairie1900.a` indique au linker où trouver la librairie compilée et comment la lier au projet final. Le paramètre `-L` spécifie le chemin de recherche des librairies, tandis que `-l:librairie1900.a` force la liaison avec le fichier d'archive statique spécifique.

Ensuite, nous introduisant une cible debug qui enrichit la capacité du Makefile. Cette cible ajoute les flags -DDEBUG -g lors de la compilation, activant les symboles de débogage et définissant la macro DEBUG. Cette fonctionnalité permet de compiler deux versions distinctes du programme sans modifier le code source : une version optimisée pour la production et une version instrumentée pour le développement et le débogage.

Finalement, le nettoyage automatique des fichiers intermédiaires dans la cible all représente une amélioration ergonomique. Les commandes `rm *.o` et `rm *.d` suppriment automatiquement les fichiers objets et de dépendances après une compilation réussie, maintenant le répertoire de travail propre et évitant l'accumulation de fichiers temporaires.

Modifications du Makefile du code de test:

Le Makefile utilisé pour compiler le code de test démontre une architecture bien pensée pour l'intégration avec la librairie. Le changement de nom du projet de robot à `exec` reflète la nature différente de ce Makefile : il ne produit plus un robot autonome, mais un exécutable de test. Cette distinction améliore la clarté du projet.

L'utilisation de `$(wildcard *.cpp)` pour les fichiers sources permet au Makefile de test de compiler automatiquement tous les fichiers de test présents, facilitant l'ajout de nouveaux cas de test sans intervention manuelle. Cette flexibilité est nécessaire dans un environnement de développement itératif où les tests évoluent constamment.

La séparation entre la compilation de la librairie et celle du code de test illustre le principe de modularité, où chaque composant peut être développé, testé et maintenu indépendamment. Le système de dépendances automatiques via les fichiers `.d` assure que seuls les fichiers modifiés sont recompilés, optimisant ainsi le temps de développement. Cette architecture facilite la collaboration en équipe et garantit la reproductibilité des compilations sur différentes machines de développement.